Telematics Platform User Guide

Flash Scrub and Read Patrol for ConMod

Document Information	
Filename	User_Guide_Flash_Scrub_and_Read_Patrol_ConMod
Status	Draft
Revision Date	Oct 11, 2022
Version	2.0
Document ID	TP_F101U-00059

Standard Proprietary Terms
THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.

© Continental Automotive GmbH 2020
Confidential and Proprietary

Table of contents

1 Introduction ... 3
1.1 Notational Conventions ... 3
2 Setup ... 4
2.1 Tools/Equipment Needed ... 4
2.2 Setup Instructions ... 4
2.3 Setup Troubleshooting ... 4
3 Features/Components ... 4
3.1 Flash Scrub ... 4
3.1.1 Overview ... 4
3.1.1.1 Platform Content Overview ... 4
3.1.1.2 Product Expectations ... 5
3.1.2 Setup ... 5
3.1.2.1 Tools/Equipment Needed ... 5
3.1.2.2 Setup Instructions ... 5
3.1.2.3 Setup Troubleshooting ... 6
3.1.3 Usage ... 6
3.1.3.1 Basic Instructions ... 6
3.1.3.2 Detailed Instructions ... 7
3.1.3.3 Testing/Error Reporting ... 11
3.1.3.4 Testing/Error Reporting ... 11
4 Known Issues ... 11
5 FAQs ... 11
6 Troubleshooting/Tips and Tricks ... 11
7 References ... 11
7.1 Abbreviations/Acronyms/Definitions ... 11
7.2 Platform Documentation ... 11
7.3 Website links ... 11
8 Revision History ... 12

© Continental Automotive GmbH 2020
Confidential and Proprietary

1 Introduction

This document provides usage information for Flash Scrub feature.

The NAND Flash scrub process is required for ensuring the integrity of the non-UBI partitions and increasing NAND longevity. The concept of flash scrub provides a software solution to refresh the read-only non-UBI partitions periodically, where the period can be configured. The refresh is done by moving data to different NAND memory locations to address the following NAND flash hardware limitations:

- Blocks could become bad at runtime
- After a fixed number of reads, a page deteriorates and loses data.

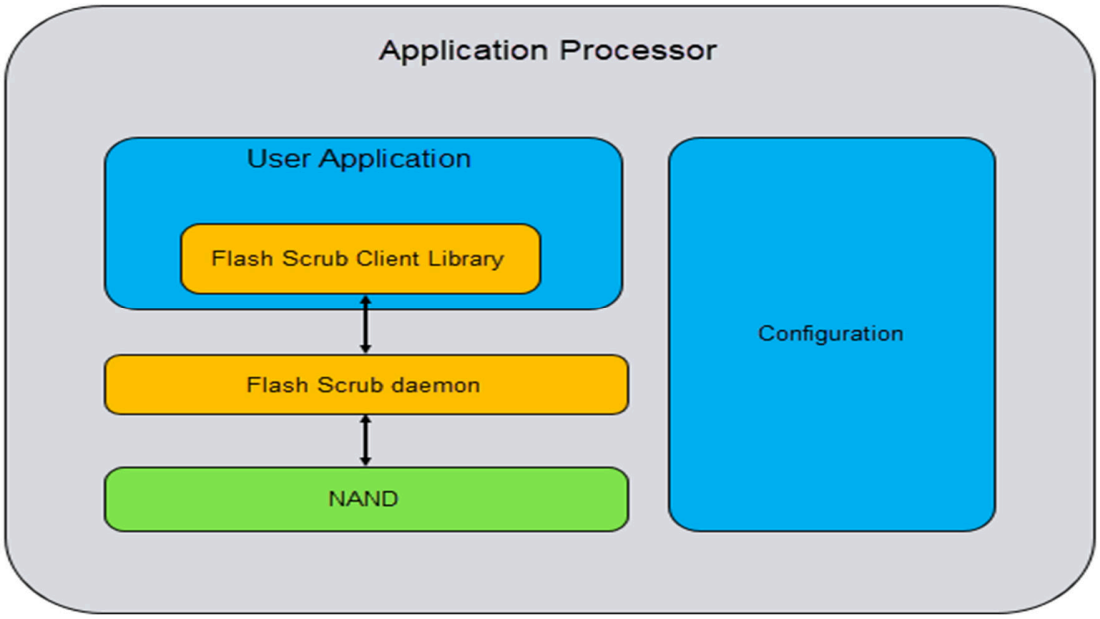


Figure 1. Flash Scrub Context Diagram

1.1 Notational Conventions

Ctrl+x	*A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button*
[ ]	*Optional elements in a command*
…..	*Indicates one of the following possibilities:* • *Additional optional arguments in a statement have been omitted* • *The preceding item or items can be repeated one or more times* • *Additional parameters, values, or other information can be entered*
-	*Line continuation: indicates that a command line or code line continues on the following line when at the end of a line*
Italic text	*Indicates important information, complete titles of manuals or variables*
Mono-spaced	*Example code or interactive screen display/output*
Numbers	*All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated*

© Continental Automotive GmbH 2020
Confidential and Proprietary

Underline blue text	*A link to an external reference (e.g., website, document)*
Modules |-> section |-> **item**	*A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)*

Diagram Legend

- Telematics Platform
- Product Specific
- 3rd Party

2 Setup

These instructions are designed to help the user of Flash Scrub to set up a project capable of using the services provided.

2.1 Tools/Equipment Needed

Not Applicable

2.2 Setup Instructions

1. Use pkg-config in the client application Makefile to link to the Flash Scrub client library.
   `LDFLAGS += `pkg-config --libs lcas-flash-scrub`
   `CFLAGS += `pkg-config --cflags lcas-flash-scrub`
2. Include Flash Scrub header file in client application source code
   `#include "flash_scrub_shared.h"`
3. Flash scrub service will not be running at boot and the unit is invoked when SCRUB service initialization api is called.
4. Security: usage of Flash Scrub APIs requires clients (user) to be a member of "swl-ua" group.

2.3 Setup Troubleshooting

Not Applicable

3 Features/Components

3.1 Flash Scrub

3.1.1 Overview

3.1.1.1 Platform Content Overview

The Flash Scrub is a daemon performing maintenance operations on the Non-Volatile Storage(s) of the system to ensure the data retention over the expected lifetime of the system.

Warning

NAND flash storage have a 10-year guaranteed data retention time in optimal conditions. Considering the automotive conditions (above 55°C ambient temperature) and the wearing, the effective guaranteed data

© Continental Automotive GmbH 2020
Confidential and Proprietary

retention time is much lower. So, maintenance operations are **MANDATORY** to extend the data retention time to the expected lifetime of the system.

The Flash Scrub daemon provides an abstraction of the underlying Non-Volatile Storage type, so that the project client application can be generic and manage different kinds of storage in the same way.

A client of the Flash Scrub can:

- trigger Read or Write refresh operations on one configured area
- get notifications of the daemon state
- get notifications on refresh operations
- retrieve the refresh status of one configured area

The Flash Scrub daemon uses persistency to store the status of the last refresh of each area, as well as the progress of a started but not completed refresh.

The Flash Scrub daemon supports different types of Non-Volatile Storages, using compilation flag raw NAND flash when FLASH_SCRUB_NAND is set to 1

3.1.1.2 Product Expectations

The following expectations must be met by application program:

1) Application should immediately interrupt scrubbing procedure if external shutdown request have been detected and restart the scrubbing process after power on.
2) Project application should ensure that none of the NAND partitions are accessed during the scrub process, except for UBI partition.
3) Project application should ensure an SoC reboot, after completing flash scrub process as partitions between MIBIB and UBI are relocated.
4) Project application shall be recommended to start flash scrubbing process just before entry to DNO and also when software download is not scheduled
5) Project application shall call **tpSCRUB_refresh** for both read and write refreshes. Write refreshes will perform the scrub for non-UBI partitions and read refreshes will cover the read patrol for the UBI partitions.

3.1.2 Setup

3.1.2.1 Tools/Equipment Needed

Not Applicable

3.1.2.2 Setup Instructions

Not Applicable

© Continental Automotive GmbH 2020
Confidential and Proprietary

3.1.2.3 Setup Troubleshooting

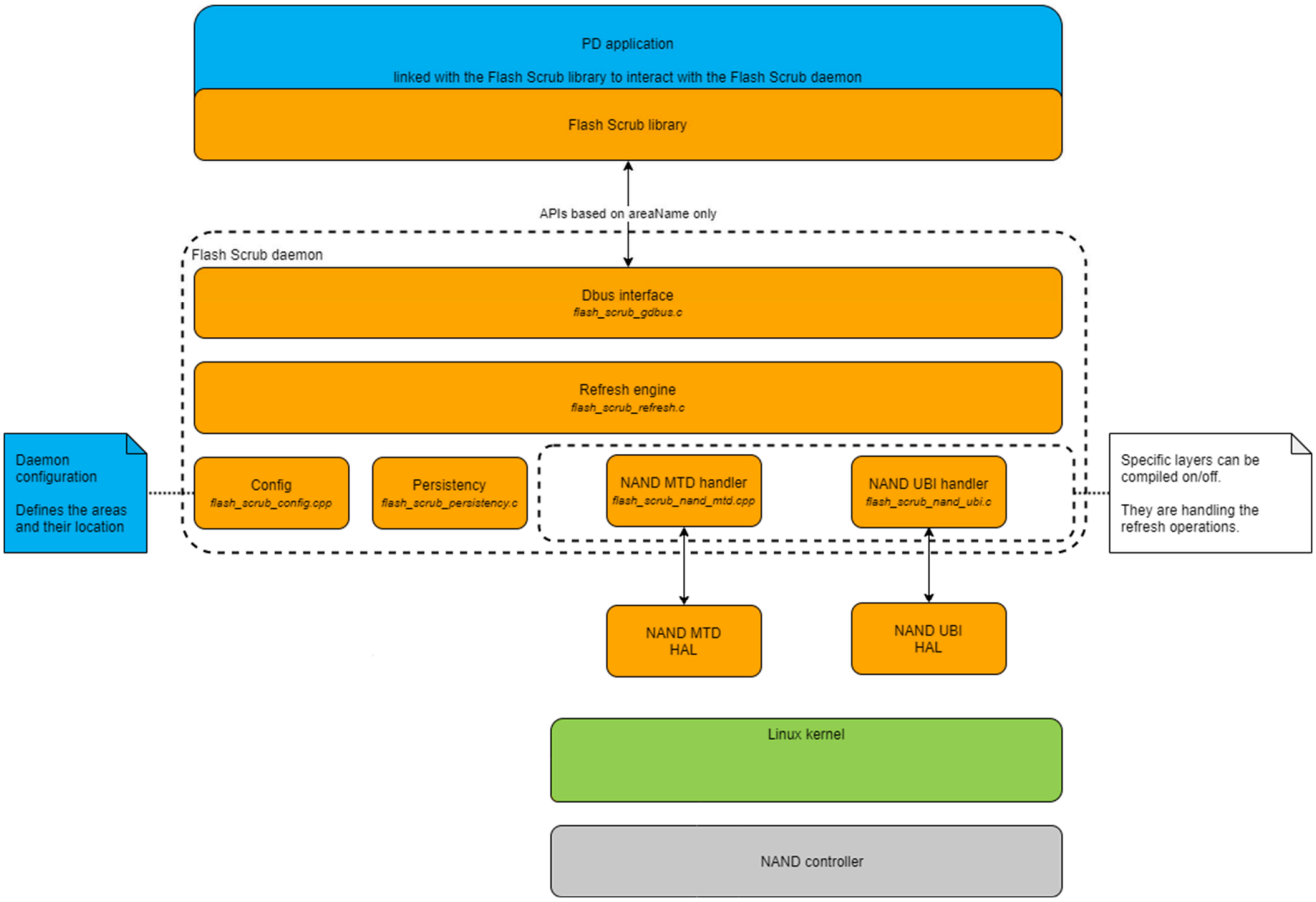
Not Applicable

3.1.3 Usage

3.1.3.1 Basic Instructions

Architecture

Here is a schema of the overall architecture of the Flash Scrub.



The Flash Scrub code can be split in 2 blocks:

- a generic part handling the daemon initialization, the generic refresh logic, the DBus communication, the configuration and the persistency

© Continental Automotive GmbH 2020
Confidential and Proprietary

- specific parts that implement the refresh operation for a dedicated non-volatile storage

The generic part can be run on any Linux system.

The specific parts may use HAL libraries to access the non-volatile storage.

Threads

The Flash Scrub is a daemon composed of 2 threads:

- the main thread is managing the Unix signals and the DBus communication
- a dedicated thread perform the refresh operations on the non-volatile storage areas.

The main thread is based on a glib main loop:

- it executes g_main_loop_run(), to handle incoming DBus messages
- on SIGTERM / SIGQUIT, the signal handler stops the glib main loop.

The dedicated refresh thread does the following:

- sleep until an autonomous refresh is due, or a refresh request arrives
- process the refresh request (self-induced for autonomous refresh, or external request)
- sleep again

3.1.3.2 Detailed Instructions

Create a Flash Scrub client

1. Include Flash Scrub header file in client application source code

```
#include "flash_scrub.h"
```

2. Wait for Flash Scrub to be ready at startup.

   a. If client application has a "hard" dependency on Flash Scrub to be running use system .service After field

```
[Unit]
After=flash-scrub.service
```


© Continental Automotive GmbH 2020
Confidential and Proprietary

b. If client application has only a single thread (or portion of process) that is dependent on Flash Scrub use TP service monitor API.

```
tpSVCMON_registerServiceStateChange(FLASH_SCRUB_SVCMON_SERVICE, FALSE, callback,
        &monitor_id);
```

3. Security: usage of Flash Scrub APIs require clients (user) to be a member of "nvm" group.
4. In the client application code, initialize the communication with the Flash Scrub daemon

```
tpSYS_ReturnCode_t sys_ret = E_SYS_IPC_RET_SUCCESS;

tpSCRUB_Error_t scrub_ret = E_SCRUB_ERROR_NONE;

sys_ret = tpSYS_initIPC(E_SYS_IPC_DBUS, NULL, NULL);

/* TODO: check sys_ret value */

scrub_ret = tpSCRUB_init();

/* TODO: check scrub_ret value */
```

Initialize Flash scrub

API/Type Description	Doxygen Path
API to initialize Flash scrub procedure	Modules |-> Flash_scrub |-> **tpSCRUB_init()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Initialize communication with the Flash Scrub service.
This function setups up IPC communication to allow the client to be able to communicate with the Flash Scrub.

De-initialize Flash scrub

API/Type Description	Doxygen Path
API to de-initialize Flash scrub procedure	Modules |-> Flash_scrub |-> **tpSCRUB_deinit()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Uninitialize Flash Scrub Client Library.
This function should be called by clients during graceful shutdown.

© Continental Automotive GmbH 2020
Confidential and Proprietary

Get Daemon Flash scrub status

API/Type Description	Doxygen Path
API to get status of Daemon Flash scrub procedure	Modules |-> Flash_scrub |-> **tpSCRUB_getStatus()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Retrieve flash scrub daemon status

Get log level of Flash scrub.

API/Type Description	Doxygen Path
API to get log level Flash scrub procedure	Modules |-> Flash_scrub |-> **tpSCRUB_getLogLevel()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Get Refresh of status Flash scrub.

API/Type Description	Doxygen Path
API to get refresh of status Flash scrub procedure	Modules |-> Flash_scrub |-> **tpSCRUB_getRefreshStatus()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Retrieve the status of a given non-volatile storage area.

Subscribe/ Unsubscribe to Flash scrub daemon events notification

API/Type Description	Doxygen Path
API to register to receive flash scrub daemon events notification	Modules |-> Flash_scrub |-> **tpSCRUB_registerDaemonEventCallback()**
API to deregister to receive flash scrub daemon events notification	Modules |-> Flash_scrub |-> **tpSCRUB_deregisterDaemonEventCallback()**
Enumeration of error response codes	Modules

© Continental Automotive GmbH 2020
Confidential and Proprietary

API/Type Description	Doxygen Path
	|-> Flash_scrub |-> **tpSCRUB_Error_t**

Subscribe/ Unsubscribe to Flash scrub of refresh events notification

API/Type Description	Doxygen Path
API to register to receive flash scrub refresh events notification	Modules |-> Flash_scrub |-> **tpSCRUB_registerRefreshEventCallback()**
API to deregister to receive flash scrub refresh events notification	Modules |-> Flash_scrub |-> **tpSCRUB_deregisterRefreshEventCallback()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Refresh of flash scrub process

API/Type Description	Doxygen Path
API to trigger the Flash scrub refresh	Modules |-> Flash_scrub |-> **tpSCRUB_refresh()**
Enumeration of error response codes	Modules |-> Flash_scrub |-> **tpSCRUB_Error_t**

Trigger a refresh of the given non-volatile storage area.

Note:

The refresh operation is only scheduled and will be performed asynchronously.

Attention:

Required Permissions: Caller must be a member of 'nvm' group to use this API.

The following log levels could be set:

Flash scrub status	Scenario
E_LOG_SEV_FATAL	Fatal log level, indicates system is unusable
E_LOG_SEV_ERROR	Error log level, indicates a non-fatal error
E_LOG_SEV_WARN	Warning log level, indicates an unexpected condition to be warned
E_LOG_SEV_INFO	Informational log level, normally logged by default
E_LOG_SEV_DEBUG	Debugging log level, only logged for debugging
E_LOG_SEV_VERBOSE	Verbose log level, only logged for debugging with many extra traces (never logged by default)

© Continental Automotive GmbH 2020
Confidential and Proprietary

These API returns **tpSCRUB_Error_t** – E_SCRUB_ERROR_NONE (0) value if success.

3.1.3.3 Testing/Error Reporting

Not Applicable

3.1.3.4 Testing/Error Reporting

Not Applicable

4 Known Issues

5 FAQs

6 Troubleshooting/Tips and Tricks

7 References

7.1 Abbreviations/Acronyms/Definitions

7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary

7.3 Website links

© Continental Automotive GmbH 2020
Confidential and Proprietary

8 Revision History

Version	Date	Description
1.0	Feb 7, 2022	Initial Release
2.0	Oct 3, 2022	Update to new robust implementation

© Continental Automotive GmbH 2020
Confidential and Proprietary